ChemiStreams Technical Guide

Created: May 27, 2025
Last Updated: March 17, 2026

Ranjot Dharni

Table of Contents

Table of Contents 1
Abstract 2
Terminology & References 3
Node.js 3
Node Package Manager (NPM) 3
Package.json 3
Tech Stack 4
Installation 6
Prerequisites 6
Node.js 6
Git 6
Download Source Code 6
Install Dependencies 7
Core Project Scripts 8
Firebase 9
Firebase Setup 9
Firebase Authentication 9
Auth Credentials 10
Next Firebase Auth Edge 11
Sign Up 12
Log In 13
Firebase Realtime 13
Structure 14
UseListener Hook 20
Cloud Storage 22
Google Drive 22
Drive Space Implementation 22
Message Types 23
Type Reference 23
Type Examples 24
Message Type 24
File Type 24
YouTube Embed Type 24
Spotify Embed Type 25
Closing Remarks 25

Abstract

ChemiStreams is a browser-to-browser chat application. **Message your friends, create group chats, and customize your profile to stand out.** Share your favorite music and videos with your friends using our **Spotify** and **YouTube** embedding features. This web app is a remake of an existing web app; the original ChemiStreams was one of the very first web apps created by the maker of ChemiStreams (a simple HTML, CSS, and JavaScript website). Years later, the creator of ChemiStreams remade this application using more robust tools to ensure the best experience for users, and create a more functional version of the web app. Enjoy chatting with friends using our easy-to-use, simple interface (all icons by Icons8). ChemiStreams is available to access here.

This guide goes over the different technologies used to make ChemiStreams, provides the related technical specifications, and breaks down how to add or maintain new and existing features to the application. Prerequisite knowledge is required to begin working with ChemiStreams. This includes an understanding of fundamental programming concepts, familiarity with different concepts of web development such as user authentication or event listeners, and a basic grasp of the different tools used to make ChemiStreams (See **Tech Stack**).

This guide is NOT a guide to the Next.js framework itself, nor is it a guide to the underlying technologies (node.js, npm, etc.). Rather, it serves as a reference to ease the future development of ChemiStreams for the original creator, or other developers aiming to contribute to or understand the ChemiStreams source code. Lastly, we emphasize that this guide is designed to support your understanding of the ChemiStreams repository. This mandates working through the source code (at least in part) either previously or concurrently with the use of this guide to supplement your understanding of the repository.

Terminology & References

Different terms and statements are used commonly in this technical guide primarily relating to the underlying technologies of the tools and framework being used. Again, this text is not a guide to the framework itself nor its underlying technologies.

For this reason, we've decided to provide the necessary sources for the official documentation where you can find more details on the commonly referenced tools and technologies. Also, many terms related to the framework and its underlying technologies may be used commonly throughout this text. Therefore, we will provide a broad oversight into what those terms mean as well.

Node.js

Node.js is a runtime, technically speaking. A runtime includes all of the software, dependencies, and files necessary to run some software. The Node.js runtime allows us to run JavaScript in a server-side environment, despite the language's original and still popular client-side implementations. Node.js provides some utilities only available on the server-side.

Node.js is commonly referred to as a language; while this isn't technically correct (JavaScript is the language, Node.js is the runtime that allows JavaScript to run on the server), this convention can often help to simplify our practical understanding of Node.js. Sometimes, Node.js will simply be referred to as Node; read the official Node.js documentation here.

Node Package Manager (NPM)

NPM (commonly written as simply npm) is the primary tool used to manage software dependencies and packages for the Node.js language. If you've ever worked on a Python project, you may have used 'pip', which is Python's primary package management tool. Read the official NPM documentation here.

Package.json

When you use the Node Package Manager, a 'package.json' file will be generated in the working directory. This file contains all of the dependencies installed in your Node project (Note: NPM can be used for projects other than just Node projects). It may also contain details and configuration data about your project.

Tech Stack

ChemiStreams was built on the following technologies:

Name	Logo	Version	Description
Node.js		22.11.0	The server-side programming language (using TypeScript superset, see below).
TypeScript		^5	The client-side programming language (superset of JavaScript).
React		^19	The front-end framework.
Next.js		^15.2.6	The full-stack framework to tie together client and server (built on React).
Tailwind		^4	A CSS library to speed up development.

Name	Logo	Version	Description
Firebase		^11.6	A SaaS used to manage user data and authentication.
Vercel		N/A	The hosting provider.
Git		2.50.1	The version and repository management tool.
GitHub		N/A	The source code repository.

Installation

Install the necessary prerequisite technologies from the following official sources.

Prerequisites

Node.js

Follow this link and follow the instructions to download Node.js to your machine. Make sure to specify your operating system using the provided options.

Git

Follow this link and select your operating system. Then, follow the provided instructions to download Git to your machine.

Download Source Code

Navigate to the directory you want to store the project in and use the following command to clone the ChemiStreams repository (you can also choose to fork your own version of the repository if you plan to integrate it with your own project or externally contribute to the existing source code):

```
git clone "https://github.com/ranjotdharni/chemistreams-2.0.git"
```

This will download the project source to your current working directory into a folder named “chemistreams-2.0”.[1]

[1] Use the form ‘git clone <repository-link> <folder-name>’ to specify a name other than ‘chemistreams-2.0’, or replace ‘<folder-name>’ with a . to download the repository contents to your current working directory.

Install Dependencies

Switch to the *chemistreams* directory and run the node package manager install command to install the necessary dependencies.

```
cd chemistreams
npm install
```

Core Project Scripts

The following section details the primary scripts the project uses both in development and in production. The Next.js framework comes with built-in commands to aid development, check your project build for errors and/or optimizations, and run a production-grade server.

Since these tools are built on top of the framework, we can technically use the *next* command to run them. For example, if we invoke the Node Package Executor (npx, comes built-in with the Node.js runtime), we can execute a script using the following syntax:

```
npx next <next-command>
```

For our project, note that the term ‘script’ refers to a configuration attribute in our *package.json* file named *scripts*. The attribute is an object where each of the object’s attributes is the script name and their values are the commands that the Node Pack Executor will run when we invoke it using the *npm* command. We prefer the node package manager convention; this makes it easier to define the commands in our *package.json* file and extend them as necessary. We can then execute a ‘script’ using the following syntax:

```
npm run <script-name>
```

Firebase

ChemiStreams uses two database services from a document-oriented NoSQL database provider, namely Firebase. Firebase is more than just a database provider, however, for our purposes we currently only need to rely on it for authentication and real-time data storage/transfer.

Firebase Setup

Before we can start working with Firebase services, we need to create a configuration(s) saturated with credentials from our Firebase console, then initialize the Firebase services that we'll be using in our app. Primarily, we need to set up the authentication service, and the realtime database service. We'll mainly be using Firestore for authentication, and Firebase Realtime database for working with chat data in real time. If you're new to Firebase, we recommend navigating to the respective sections for Firebase authentication and Firebase Realtime which provide the relevant links to the documentation for each of these services (refer to the database entity relationship diagram that supplements this guide for a visualization of how these services work together).

To initialize the Firebase services we'll be using, we need access to our Firebase app specifically designated for ChemiStreams, the Firebase database our app is connected to for handling chat data, and methods for working with Firebase authentication. All of these services are configured in the Firebase console.

The various files related to Firebase setup and initialization in our app can be found in the directory *~/chemistreams/src/lib/auth*. We highly recommend scanning through the files in this directory as they are both small files which only initialize the necessary Firebase services, and they may serve to give a more practical insight into what the following sections are referring to.

Firebase Authentication

First, the ChemiStreams service uses Firebase authentication to manage in-house users, and third-party providers (Google & GitHub). Firebase authentication is built on top of the default Firebase database known as Firestore. Firestore stores data hierarchically in a structure called a document. When we query Firestore, the service returns nested data in JSON-like structure. This makes the data very easy to work with on our front end because we can bypass certain intermediary steps like serialization

and deserialization. You can read more about Firebase authentication services here and about Firestore here.

Auth Credentials

Sensitive credentials needed to access the Firebase API are stored as environment variables. The Next.js framework imposes a restriction on public (accessible on client) and private (can only be read in server components) environment variables. All environment variables are private by default; a variable is made public when prefixed with *NEXT_PUBLIC_* (case-sensitive). If you're connecting your own fork of the project, saturate the following variables with their corresponding value from your Firebase console.

Name	Variable	Sensitive	Description
Firebase Private Key	FIREBASE_ADMIN_PRIVATE_KEY	Yes	The private key used to access your Firebase app.
Firebase Client Email	FIREBASE_ADMIN_CLIENT_EMAIL	Yes	The admin email associated with your Firebase app.
Previous Key in Rotation	AUTH_COOKIE_SIGNATURE_KEY_PREVIOUS	Yes	The previous key used in a rotating scheme for signing cookies.
Current Key in Rotation	AUTH_COOKIE_SIGNATURE_KEY_CURRENT	Yes	The current key in use for a rotating scheme when signing cookies.
Authentication Cookie Name	AUTH_COOKIE_NAME	Optional	The name of the authentication cookie.

Name	Variable	Sensitive	Description
Secure Cookies Flag	USE_SECURE_COOKIES	Optional	Whether secure cookies are used. Should be true in production.
App ID	NEXT_PUBLIC_FIREBASE_APP_ID	No	Identifies your Firebase app.
API Key	NEXT_PUBLIC_FIREBASE_API_KEY	No	Used to track usage and billing. Not used for sensitive data access.
Storage Bucket	NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET	No	Identifies your Firebase app's storage partition.
Authentication Domain	NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN	No	Identifies your Firebase app's subdomain.
Sender ID for Messaging	NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	No	Identifies the app authorized to communicate with the client.
Project ID	NEXT_PUBLIC_FIREBASE_PROJECT_ID	No	Identifies your Firebase project.

Next Firebase Auth Edge

Our implementation of Firebase authentication relies on the provided features for session-based authentication using cookies. We can manually configure our app to work with and handle cookies, however, we can speed up development efforts by relying on the recommended library for handling cookie-based authentication with Firebase and Next.js, namely Next Firebase Auth Edge.

The Next Firebase Auth Edge library takes care of the headache that comes with manually handling, encrypting, and validating authentication credentials stored in cookies. The library uses best-practice conventions for working with our app's and users' sensitive credentials such as encrypted cookies and rotating keys for cookie signatures. Once a user is successfully authenticated, we can use the methods provided by the Next Firebase Auth Edge library in our server middleware to handle successful and unsuccessful auth validation on subsequent requests. You can read about the library in more depth here.

Sign Up

A user can create an account with ChemiStreams directly by selecting the sign up option on the login page. The `SignUpPage` component (located in the directory *~/chemistreams/src/components/signup*) handles validating the details input by the user when creating a new account.

Once validated, first on the client, the details are passed to a server action that handles creation of the new user (server actions can be found in the file *~/chemistreams/src/lib/utils/server.ts*). The `signUpAction` function is invoked during new user creation; it follows a simple flow to create a new user.

1. Check if the user already exists. (if so, return a `SIGNUP_FAILURE_ERROR`)
2. If the user doesn't exist, invoke the `createUserWithEmailAndPassword` method provided by Firebase authentication, and pass in the necessary credentials.
3. Create the corresponding record in the Firebase Realtime database so it can refer back to users as necessary (See **Firebase Realtime**).
4. Let Next Firebase Auth Edge handle authentication cookies by invoking the provided `refreshCookiesWithIdToken` method.
5. Redirect the user to the home page upon success.

If successful, the user's account will be created in the Firestore instance connected to Firebase authentication, and our Firebase Realtime instance will have a reference to that user in Firestore via their unique username.

Log In

There are two ways to log in to ChemiStreams:

1. Use an existing account made directly by signing up with ChemiStreams.
2. Use an existing Google or GitHub account to log in to ChemiStreams.

Method 1

Referring back to our server actions, the `loginAction` method is invoked when logging in with an existing account made directly with ChemiStreams. This method generally follows the same behavior as the `signUpAction` method, except it does not create any new user details in any of our connected database services. It also uses the `signInWithEmailAndPassword` method to log a user in rather than the `createUserWithEmailAndPassword` method to sign a user up.

Method 2

When we use an existing Google or GitHub account to log a user in, we must check if the user has already authenticated using a Google or GitHub account before. If it's their first time logging in with a provider, we need to create the necessary user details in our Firebase Realtime instance, however we do not need to update anything in our Firestore instance. This is because Firestore is used for in-house authentication details; these details are handled by the third-party authentication provider in question, and we can retrieve them simply by invoking the ID token generated when the user first logs in with a provider.

When a user clicks on a provider login button in the `LoginPage` component, the page redirects them to the corresponding provider's login page. Once authenticated there, the provider will redirect them back to our login page with an ID token. We can then exchange this ID token for the necessary authentication credentials simply by invoking the same `refreshCookiesWithIdToken` method used in our sign up flow provided by the Next Firebase Auth Edge library.

Note that provider login for both Google and GitHub require configuration for our app in both the Firebase console (for Google, Firebase was created and is maintained by Google) and the GitHub developer settings respectively.

Firebase Realtime

Firebase Realtime works by allowing multiple users to maintain concurrent connections to the database (only with access to the specific data they're interested in or allowed to

view). After a connection is established, the client holds references to specific data in the database; Firebase Realtime will reflect changes to the referenced data in real time (supporting 200,000 concurrent connections with response times below ~10ms).[2]

To work with a Firebase database in a client app, we need to create a reference to the database instance in question. We can do this by invoking the `getDatabase` method provided by the Firebase library. Refer to the *~/chemistreams/src/lib/auth/firebase.ts* file on how to initialize a reference to our Firebase Realtime instance.

Structure

Once again, Firebase Realtime stores and returns data as a JSON object(s). If we query the entire database, the response is the root object which is a JSON object with attributes that nest other JSON objects. Note that we should limit data nesting and prefer expanding each layer of the root object horizontally rather than nesting more attributes vertically (keep data as flat as possible).

The following table is reference for the different objects and their attributes as they relate to our application[3]:

Name	Attribute	Type	Description
Chat Groups	groups	List	A list of chat IDs of all group chats.
Chat Group	groups.<chat_id>	List	A list of user IDs of all the users part of the top-level group.
Chat Group Member	groups.<chat_id>.<user_id>	Boolean	Necessary placeholder; always true. The key of this value is a user ID.

[2] Firebase Realtime Limits, Firestore vs Firebase Realtime.
[3] In Firebase Realtime, lists are not arrays. Rather, a *List* type is an object with a repetitive set of attributes where each attribute is an unique ID that maps to the same repeated type of object.

Name	Attribute	Type	Description
Chat Messages	messages	List	A list of all chat IDs.
Chat Message List	messages.<chat_id>	List	A list of all message IDs pertaining to the top-level chat.
Chat Message	messages.<chat_id>.<message_id>	Object	The content and details of a specific chat message.
Message Content	messages.<chat_id>.<message_id>.content	String	The actual text in a sent message.
Message Sender	messages.<chat_id>.<message_id>.sender	String	The user ID of the user that sent a particular message.
Message Timestamp	messages.<chat_id>.<message_id>.timestamp	UNIX Timestamp	The exact time when a particular message was sent.
Message Type	messages.<chat_id>.<message_id>.type	Number	The type of message being sent.
Chat Metadatas	metadata	List	A list of all chat IDs.
Chat Metadata	metadata.<chat_id>	Object	Metadata for a particular chat.

Name	Attribute	Type	Description
Chat Alias	metadata.<chat_id>.alias	String	The display name for a particular group chat.
Chat Creator	metadata.<chat_id>.creator	String	The user ID for the creator of a particular chat.
Chat Recipient	metadata.<chat_id>.to	String	The user ID for the recipient of a particular direct chat.
Is Group Chat	metadata.<chat_id>.group	Boolean	Indicates if a particular chat is a group chat. Always true.
Usernames	usernames	List	A list of all users' usernames.
Username	usernames.<username>	String	The user ID assigned to a particular username.
Users	users	List	A list of all users' comprehensive details.
User	users.<user_id>	Object	A set of details for a particular user.
User Badge	users.<user_id>.badge	Object	A particular user's badge details.

Name	Attribute	Type	Description
User Badge Animation Code	users.<user_id>.badge.animationCode	String	Indicates which animation to apply to a particular badge image.
User Badge Code	users.<user_id>.badge.badgeCode	String	Indicates which badge image to apply to a particular user.
User Bio	users.<user_id>.bio	String	A particular user's bio.
User Chats	users.<user_id>.chats	List	A list of chat IDs of all the chats a particular user is a part of.
User Chat	users.<user_id>.chats.<chat_id>	String	The message ID of the most recent message sent in a particular chat.
Display Name	users.<user_id>.name	String	A particular user's set display name.
Profile Picture	users.<user_id>.pfp	Object	A particular user's profile picture details.
Profile Picture ID	users.<user_id>.pfp.id	String	The resource identifier for a particular user's PFP.
Profile Picture Link	users.<user_id>.pfp.link	String	The URL where a particular user's PFP can be found.

Name	Attribute	Type	Description
Profile Picture Mime Type	users.<user_id>.pfp.mime	String	A particular PFP's mime type.
Profile Picture Space	users.<user_id>.pfp.space	String	The drive space ID where a particular PFP is stored.
Profile Picture File Type	users.<user_id>.pfp.type	String	The file type of a particular PFP.
User Status	users.<user_id>.status	Boolean	Indicates if a particular user is currently online.
Is User Typing	users.<user_id>.typing	String	Indicates if a particular user is currently typing.
Username Reference	users.<user_id>.username	String	A particular user's username.

The following JSON object is an example of ChemiStreams' Firebase Realtime instance's structure; the outer-most object is the root object:

```
{
  "groups": {
    "92bd153e-9563-489f-8133-fb1abae0a632": {
      "DpvB0lhaJiORXL3dDHZI2Fpgo352": true,
      "La3NKxCWxDQt3vK014yfbj3wAMm2": true,
      "PpWlE5xOueYysRyvUvLm5IJyvtg2": true
    }
  },
  "messages": {
    "92bd153e-9563-489f-8133-fb1abae0a632": {
      "-ORbGTb_r7Ih_dLUIqxz": {
        "content": "Wassup buckos",
        "sender": "La3NKxCWxDQt3vK014yfbj3wAMm2",
        "timestamp": 1748710582830,
        "type": 0
      },
      "-ORbGi4uRcmSe4uNmU3N": {
        "content": "your badge is going brrrrrr",
        "sender": "DpvB0lhaJiORXL3dDHZI2Fpgo352",
        "timestamp": 1748710646230,
        "type": 0
      }
    }
  },
  "metadata": {
    "92bd153e-9563-489f-8133-fb1abae0a632": {
      "alias": "Losers Chat",
      "creator": "La3NKxCWxDQt3vK014yfbj3wAMm2",
      "group": true
    }
  },
  "usernames": {
    "ranjotdharni1": "PpWlE5xOueYysRyvUvLm5IJyvtg2",
    "tgreene1": "La3NKxCWxDQt3vK014yfbj3wAMm2",
    "thorton1": "DpvB0lhaJiORXL3dDHZI2Fpgo352"
  },
  "users": {
    "DpvB0lhaJiORXL3dDHZI2Fpgo352": {
      "badge": {
        "animationCode": "NULL",
        "badgeCode": "BADGE_ELDENRING"
      },
      "bio": "Gimme Coffee Bruv",
      "chats": {
        "92bd153e-9563-489f-8133-fb1abae0a632": "-ORbGpaqPCNgAK5tJwzw",
        "d7da2fc3-c247-4f69-bb78-12fcd24d26e8": "-OT6OPiWTlggOBhxQmGu"
      },
      "name": "Tim Horton",
      "pfp": {
        "id": "1zLuKbmCDPO68zcmh2M14uLEehOHKWlHU",
        "link": "https://drive.google.com/uc?export=view&id=1234",
        "mime": "image/png",
        "space": "10090",
        "type": "png"
      },
      "status": false,
      "typing": "",
      "username": "thorton1"
    }}}
```

UseListener Hook

Firebase Realtime relies on listeners to reflect changes in the database on the client-side in real time. In general, this is the flow a chat will follow:

1. A user logs in to ChemiStreams; this establishes a connection with the Firebase Realtime instance (the listener).
2. The user sends a message to another user (or group chat); this makes an update to that chat in the database.
3. The message recipient's listener will see the update to that data and reflect the change on the recipient's client.

We use listeners commonly throughout ChemiStreams in this same manner, for example, to send messages, to track when users go online/offline and to detect when a user is typing in a chat. Since the listeners' functionality is so common, we can leverage the React framework's custom hooks feature to create a reusable abstraction for implementing listeners where we need them.

This is where we can leverage the `useListener` custom hook.

Parameters

reference – A reference to the data we want to attach a listener to.
config – A configuration object. See next subsection for more detail.
precondition – A function that runs before the listener attaches.
postcondition – A function that just before the listener attaches.

Config Parameter

The *config* parameter is an object of the following type:

```
export interface UseListenerEventDetails {
   callback: (snapshot: DataSnapshot) => void
   errorCallback: (error: Error) => void
}

export interface UseListenerConfig {
   value?: UseListenerEventDetails
   added?: UseListenerEventDetails
   changed?: UseListenerEventDetails
   removed?: UseListenerEventDetails
   moved?: UseListenerEventDetails
}
```

Listeners are configured to tell the client a specific type of update occurred. Therefore, we need to specify which type of update(s) (or event) a particular listener should listen for.

Each listener config can have up to 5 (and at least 1) different event types that can satisfy ChemiStreams' use cases. Each event in the above type definition is an abstraction that maps to its corresponding Firebase Realtime listener attachment function in Node.js (use the hyperlinks below to read more about how each event behaves):

value: `onValue`
added: `onChildAdded`
changed: `onChildChanged`
removed: `onChildRemoved`
moved: `onChildMoved`

When we specify an event to listen for in the configuration passed to our custom hook, we must specify a callback function that runs when the event is triggered (this callback function takes a 'snapshot' of the update that occurred), and an error callback function to handle cases where some error occurred when attempting to detect or read an update to the data that a particular event is attached to.

Cloud Storage

ChemiStreams needs to be able to store media files. However, due to the project's limited scope and resources, we've limited support for media files to images alone.

Nevertheless, cloud storage is costly and an alternative is necessary given the limited resources backing the app to store and handle media file uploads. This is where we can take advantage of cheap resources provided by various cloud storage providers to create a working cloud store for media files. The general idea is that we can have multiple accounts (dedicated to ChemiStreams) with available, cheap (ideally free) cloud storage space to create a conglomerate cloud store for media files.

Google Drive

Google Drive offers 15GB of free cloud storage per account. ChemiStreams currently has a single dedicated Google Drive account that acts as the cloud store for storing and serving media files. However, the conglomeration system described in this section is built on the idea of extensibility as needed.

We use the Google Drive API in conjunction with service accounts to avoid UI authentication for each API call. Service accounts can use pre-generated service account keys to repeatedly access the Google Drive accounts they're associated with.[4] Account keys are stored as private environment variables (see **Auth Credentials**) and only accessible on the server-side.

Drive Space Implementation

ChemiStreams' use of conglomerate Google Drive accounts is referred to as the **Drive Space** system. The Drive Space system uses a round robin approach for multiple Google Drive accounts. This is to say that with a list of accounts, the system will upload a single item to a single account and then upload the next incoming item to the next account in the list, moving cyclically when it reaches the end of the list.

Google Drive can generate web view links for retrieving uploaded images. This means each uploaded item does not need to be looked up in its corresponding Drive Space every time we want to read from Google Drive. We can simply upload an image using

[4] From the official Google documentation: service account keys are a security risk if not managed correctly…If you must authenticate with a service account key, you are responsible for the security of the private key and for other operations…

the Drive Space system, then retrieve it as many times as necessary using the generated web view link.[5]

Each Drive Space is identified by a **Drive Space ID** which is in the following format:

s00<position_in_account_list>

The implementation files for the Drive Space system can be found in the directory *~/chemistreams/src/lib/drive*.

Message Types

ChemiStreams has different types of messages you can send. Each message type exhibits a specific structure when stored in Firebase Realtime. Each type has a specific type code; this code is attached to each sent message when it's stored in the database and can be used to identify a message on subsequent reads. To find all available message types and codes, refer to the file *~/chemistreams/src/lib/constants/server.ts*.[6]

Type Reference

Name	Type Code	Description
Message Type	0	A basic text message.
File Type	4	An image file.
YouTube Embed Type	6	An embedded YouTube video. YouTube embed implementation follows the official documentation found here.
Spotify Embed Type	5	An embedded Spotify resource. Spotify embed implementation follows that official documentation found here.[7]

[5] Note: For subsequent writes to the Drive Space system (i.e. deleting an uploaded item), we must look up the specific Drive Space that an item was uploaded to.
[6] Note: You may notice that *server.ts* contains message types that aren't implemented anywhere; these are future considerations.
[7] Notice that the HTML code grabbed from Spotify is always just an iFrame with the Spotify URI to the resource in question. This means we really only need a Spotify URI to recreate an embedding.

Type Examples

Message Type

```
{
        "sender": "014yfbj3w...",
        "type": 0,
        "content": "Wassup buckos",
        "timestamp": 1748710582830
}
```

File Type

```
{
        "sender": "sdUH7V54f...",
        "type": 4,
        "link": "https://...",
        "space": "s001",
        "fileId": "4gY6f7...",
        "timestamp": 1748710582830
}
```

YouTube Embed Type

```
{
        "sender": "aJiORXL3d...",
        "type": 6,
        "youtubeId": "djH65fj77iV...",
        "timestamp": 1748710582830
}
```

Spotify Embed Type

```
{
        "sender": "014yfbj3w...",
        "type": 5,
        "spotifyId": "djH65fj77iV...",
        "resourceType": "track",
        "timestamp": 1748710582830
}
```

Closing Remarks

Before we conclude, we want to once again point out that this guide is not maximally comprehensive as it was never intended to be (also, certain details were omitted due to security concerns, i.e. database rules configuration). Ideally, you should go through both the ChemiStreams source code AND web app to supplement your reading here. Also, refer to the supplementary database entity relationship diagram alongside this guide for a deeper visual insight into how the app's databases are structured in conjunction.

We hope to expand on this guide as/if ChemiStreams grows and have done so up to the *Last Updated* date marked on the cover page. If you find any errors in this guide or want to make your own contributions, we encourage opening a pull request detailing your concern(s) and/or proposal(s).

Thank you for your time in reading through this guide and may it serve you well in your endeavor(s) or purpose(s) behind doing so in the first place.

No I don't have a receipt, I stole the floor model!

- Roger Smith, American Dad